IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

12.

Big Data II: Data Warehousing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are

PART I

Collection and Preparation
Lec 2-5

PART II

SQL
Lec 6-8

NoSQL
Lec 9-10

Big Data
Lec 11-13

PART III

Monitoring and Updates
Lec 14-17

Recap: Distributed Batch Processing

A distributed program

```
(sales.groupBy("product_id")
      .sum("amount")
      .show())
```

A distributed program still needs infrastructure, storage and a way to run again

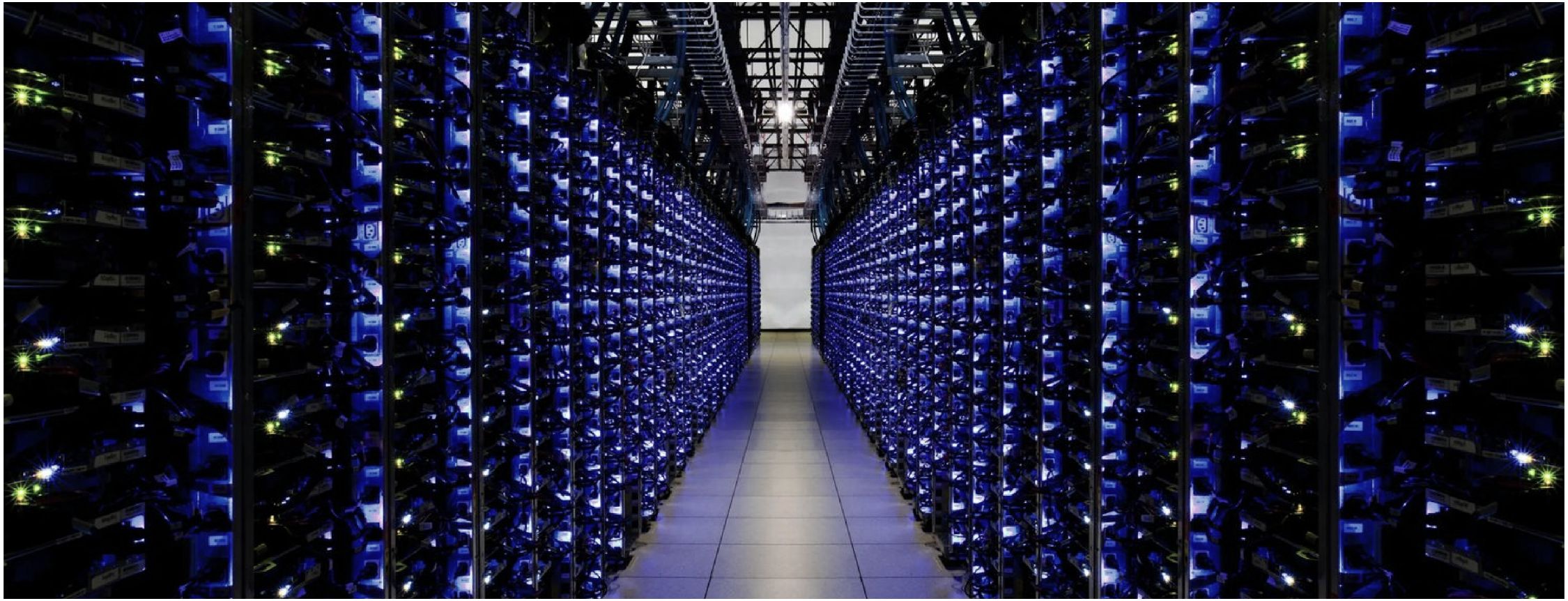
Outline: Big Data II

01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

01 · Cloud and Serverless

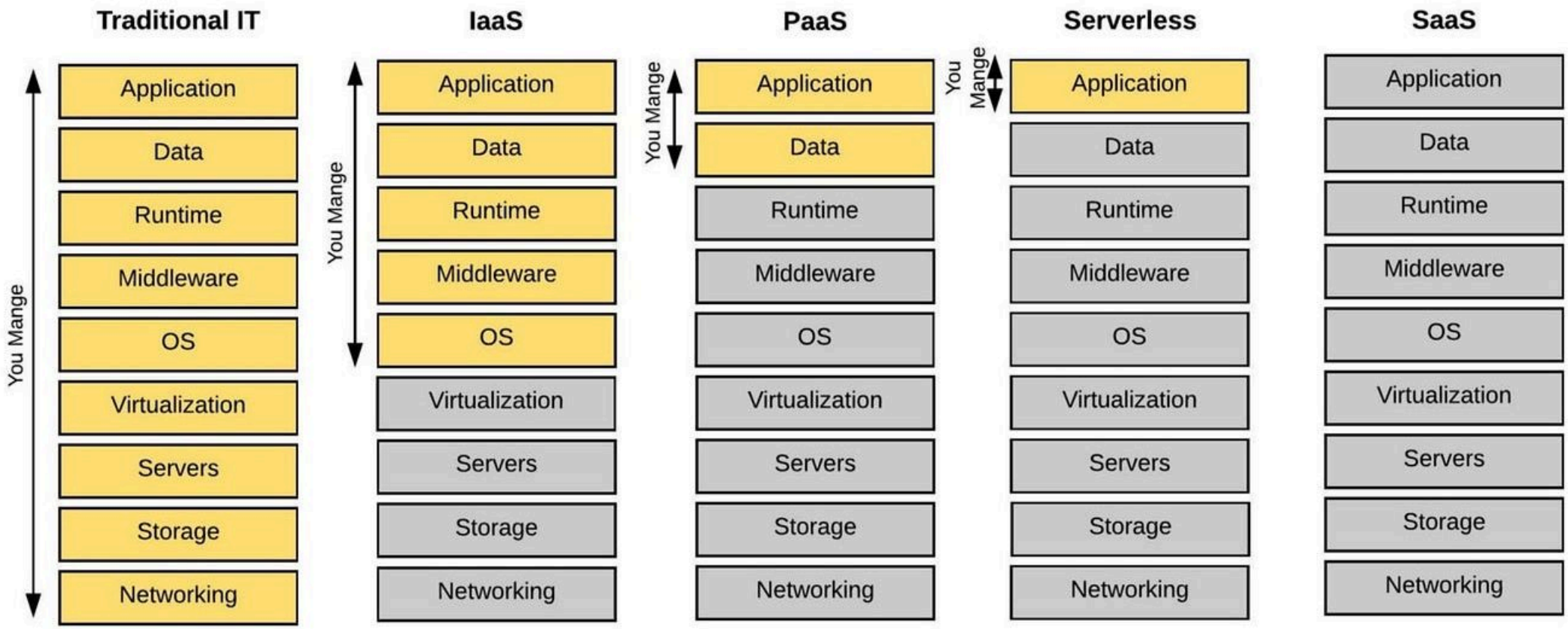
01	**Cloud and Serverless**	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Managed Infrastructure



The cloud still runs on physical servers that somebody must operate

Cloud Service Models



Moving right transfers more infrastructure operations to the provider

Serverless Computing

Serverless transfers server provisioning and operation to a managed service

Provider operates

- Servers, power and cooling
- Scaling the service
- Storage replication and recovery

You still decide

- Tables and their meaning
- SQL queries and their cost
- Who may access which data

The provider operates infrastructure while the user works with data and queries

Milestones in Managed Analytics

2004
MapReduce paper
batch on clusters

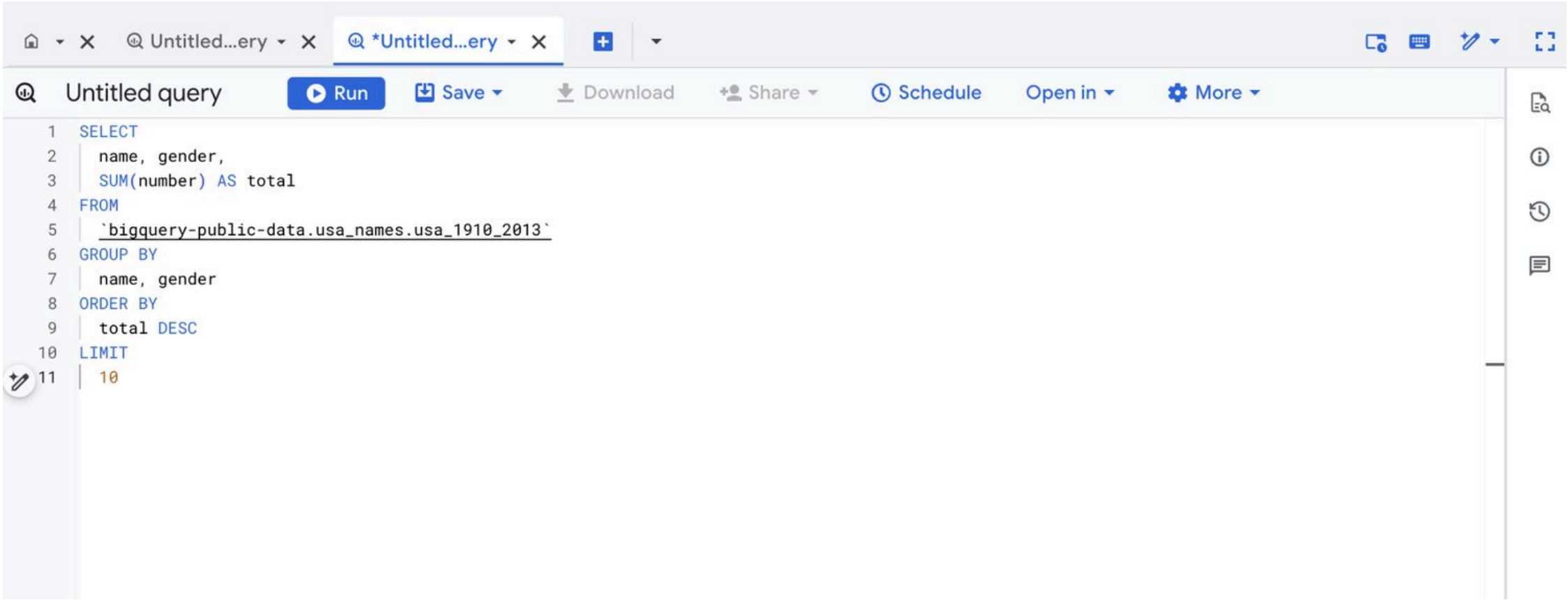
2009
Spark at Berkeley
reusable chains

2010
BigQuery preview
SQL as a service

Today
managed compute
and shared storage

Analytics shifted from operating clusters to submitting SQL to a service

BigQuery as a Managed Service



Analysts submit SQL while the service manages distributed execution

Spark and BigQuery

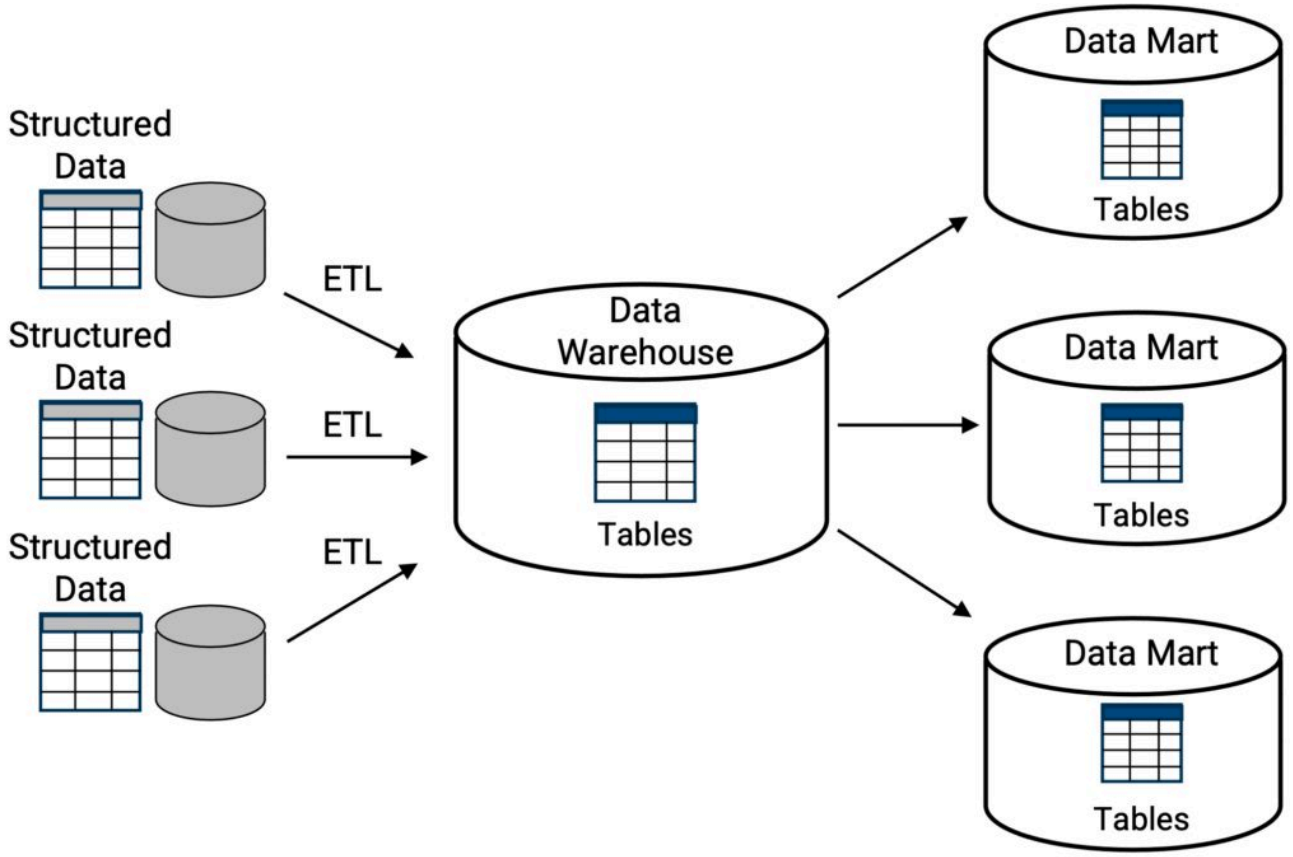
Tool	User expresses	Typical responsibility
Spark	A distributed program	Choose transformations and execution environment
Managed Spark service	A Spark program	Use a provider-managed execution environment
BigQuery	An analytical SQL query	Define tables and the analytical question

An execution engine and a managed analytical service solve different parts of the workload

02 · Analytical Warehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	**Analytical Warehouses**	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

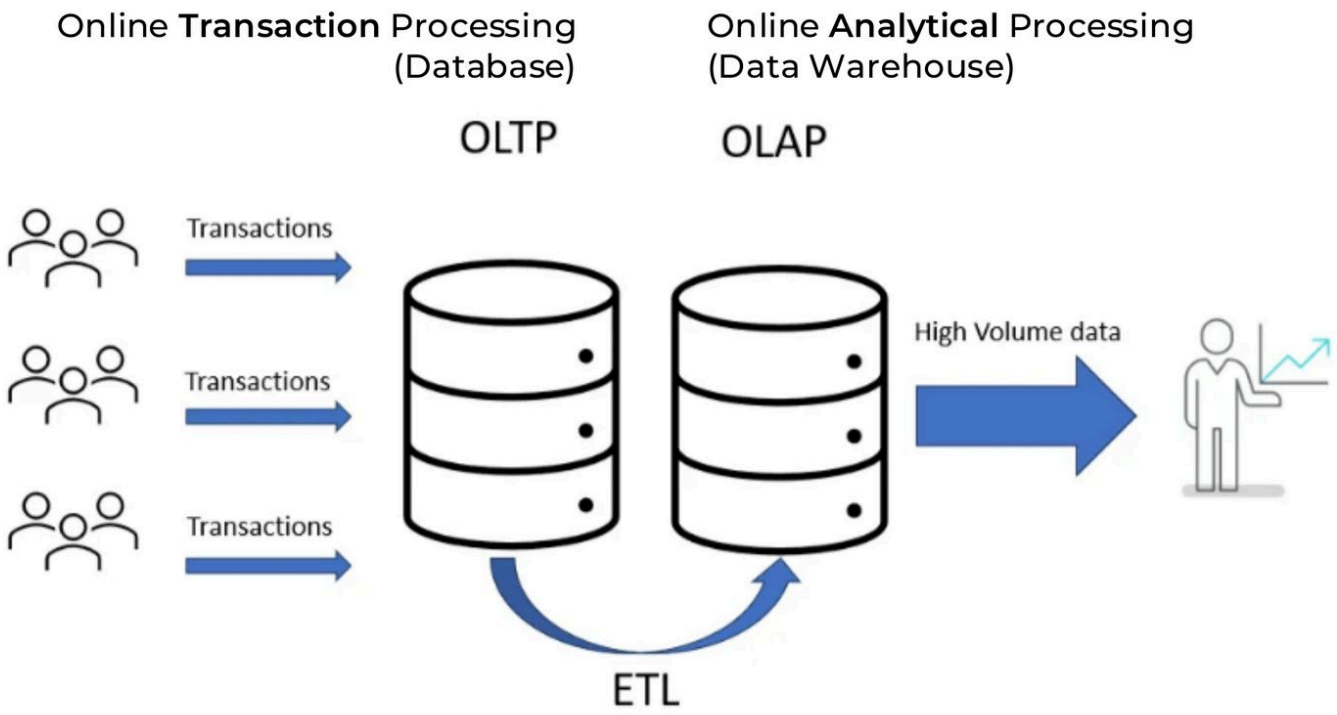
Why an Analytical Warehouse



Shared analytical tables connect multiple sources to consistent departmental reports

OLTP and OLAP

Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP)



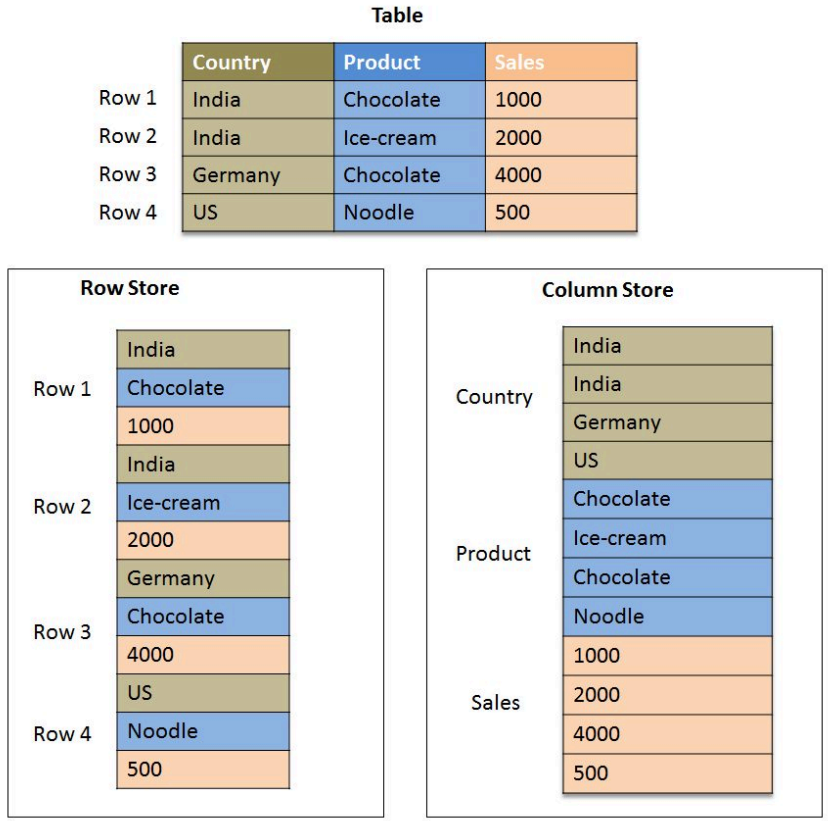
Small transactional updates and large analytical scans place different demands on storage

The Four-Sale Example

sale_id	sale_date	product_id	amount
1	2025-10-02	P1	300
2	2025-10-02	P2	200
3	2025-10-03	P1	400
4	2025-10-03	P2	100

Four sale events can support more than one analytical grouping

The Same Values on Disk



Twelve values retain their meaning while their physical order changes

Sales Scan in Row Layout

sale_id	date	product	amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

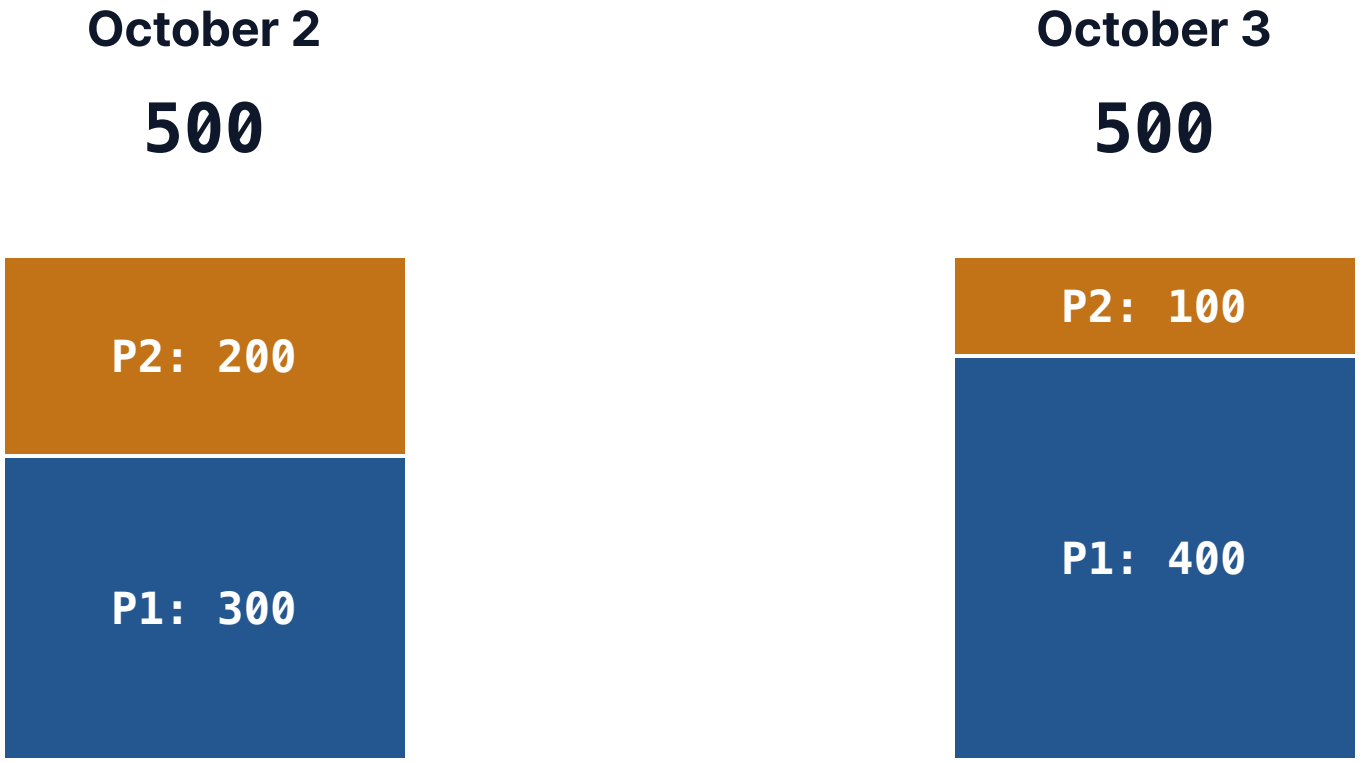
This full-row scan reads sixteen values to compute a sum of four amounts

Sales Scan in Column Layout

sale_id	date	product	amount
1	Oct 2	P1	300
2	Oct 2	P2	200
3	Oct 3	P1	400
4	Oct 3	P2	100

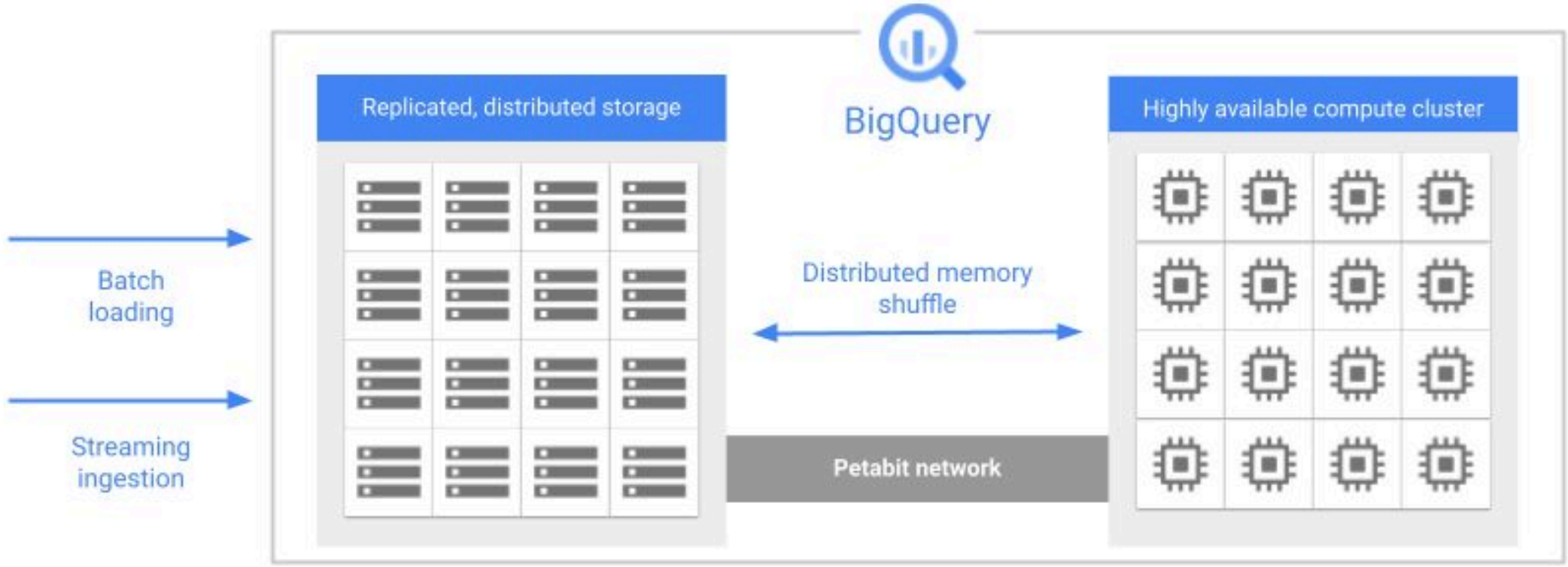
The column scan reads four amounts and returns the same total of 1000

Date-Partitioned Sales



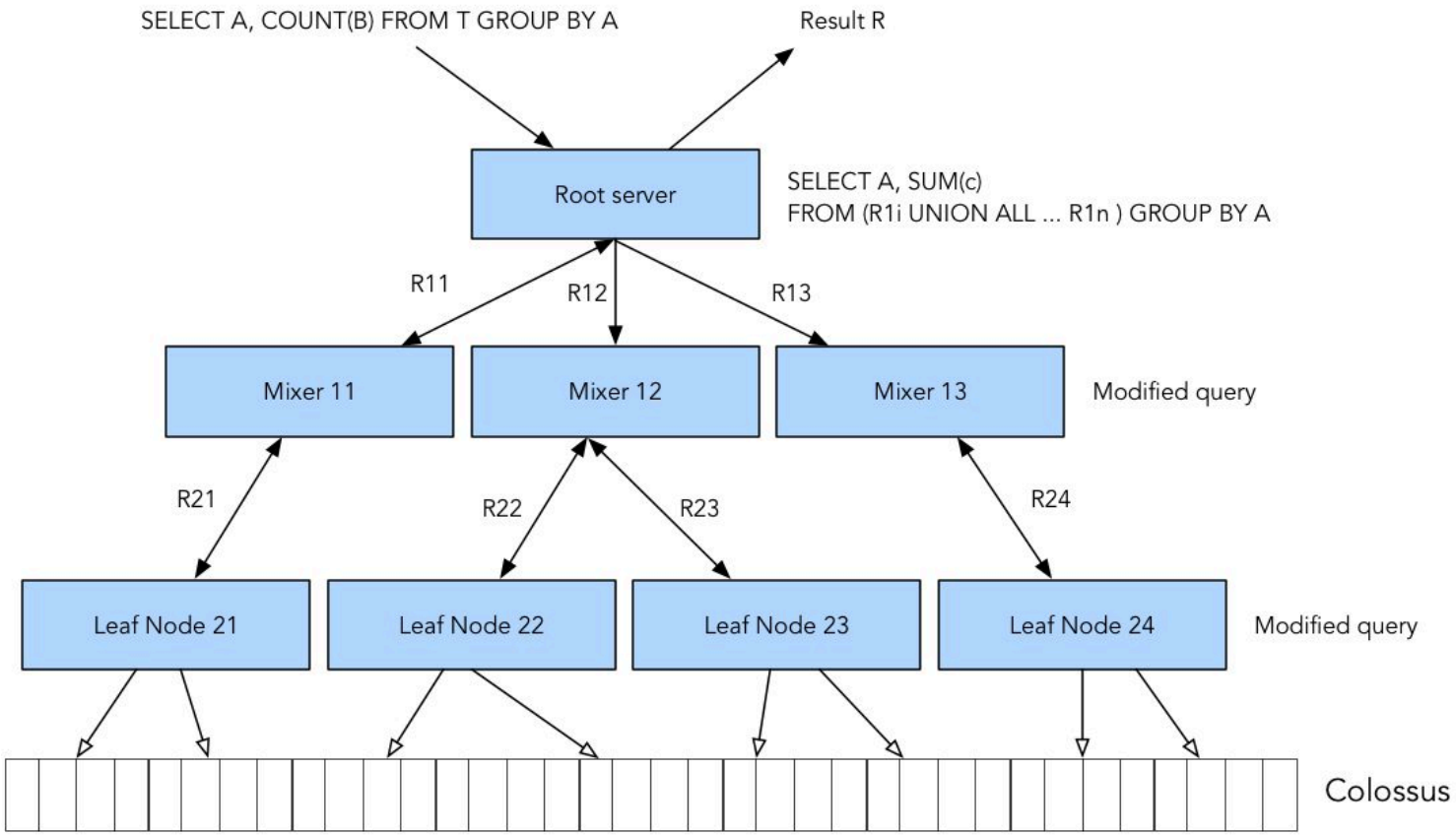
A date filter can skip unrelated partitions instead of scanning all sales

BigQuery Storage and Compute



Persistent table storage and query execution are managed as separate layers

Distributed Query Execution



Workers read portions of the table and combine partial results

Partial Sums for Four Sales

Worker A

$$300 + 200 = \mathbf{500}$$

Worker B

$$400 + 100 = \mathbf{500}$$

Combined total

$$500 + 500 = \mathbf{1000}$$

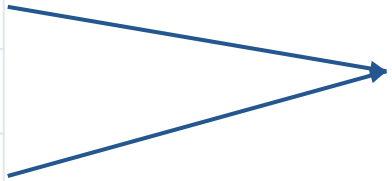
Two partial sums produce the same total as one scan over all four sales

Facts and Dimensions

A fact records an event or measurement; dimensions describe its context

fact_sales: one row per sale

sale_id	product_id	amount
1	P1	300
2	P2	200
3	P1	400
4	P2	100



dim_product: one row per product

product_id	name
P1	Chocolate
P2	Ice-cream

product_id links a sale to its name

Two sales of P1 share one product description

Grain of an Analytical Table

Sale detail

one row per sale

1	P1	300
2	P2	200
3	P1	400
4	P2	100

By product

one row per product



By date

one row per date



Changing the grouping changes what one result row represents

Demo: Analytical Questions

Local SQLite exercise: the analytical query, rather than BigQuery infrastructure

1. Return total sales for each product name
2. Keep only October 3 and compare the product totals

```
-- write your query here, then Run
```

QUERY RESULT

Write a query on the left, then Run

Answer: October 3 Product Sales

```sql
SELECT p.name,SUM(s.amount) AS total
FROM fact_sales AS s
JOIN dim_product AS p USING(product_id)
WHERE s.sale_date='2025-10-03'
GROUP BY p.product_id,p.name ORDER BY p.product_id;
```

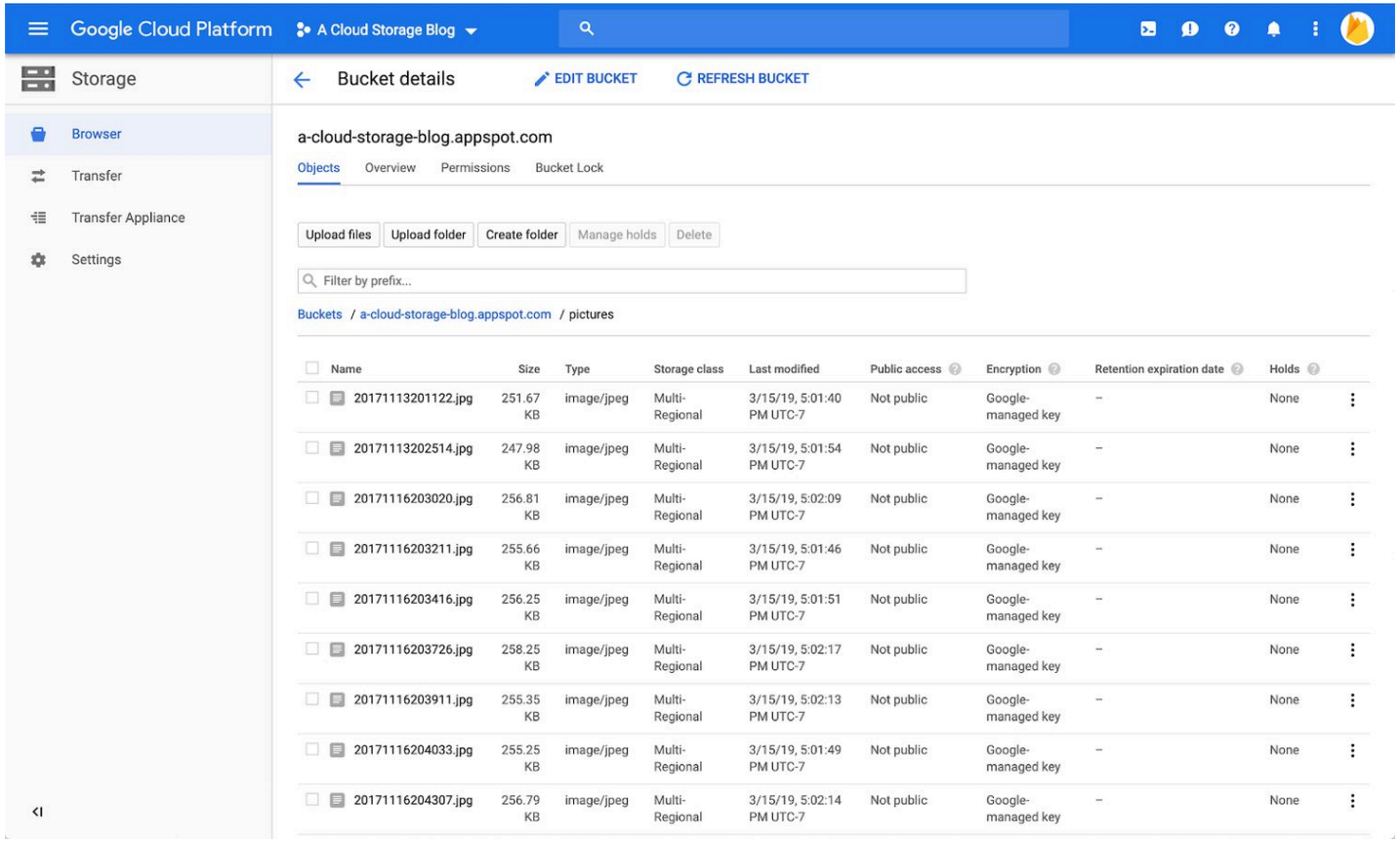
name	total
Chocolate	400
Ice-cream	100

The date filter changes the contributing sales, while each row remains one product

03 · Lakes and Lakehouses

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	**Lakes and Lakehouses**	Where do files become tables?
04	Data Orchestration	Which task runs after which, and safely again?

Why Keep a Data Lake



Source files can remain in object storage before analytical tables are defined

Schema on Read

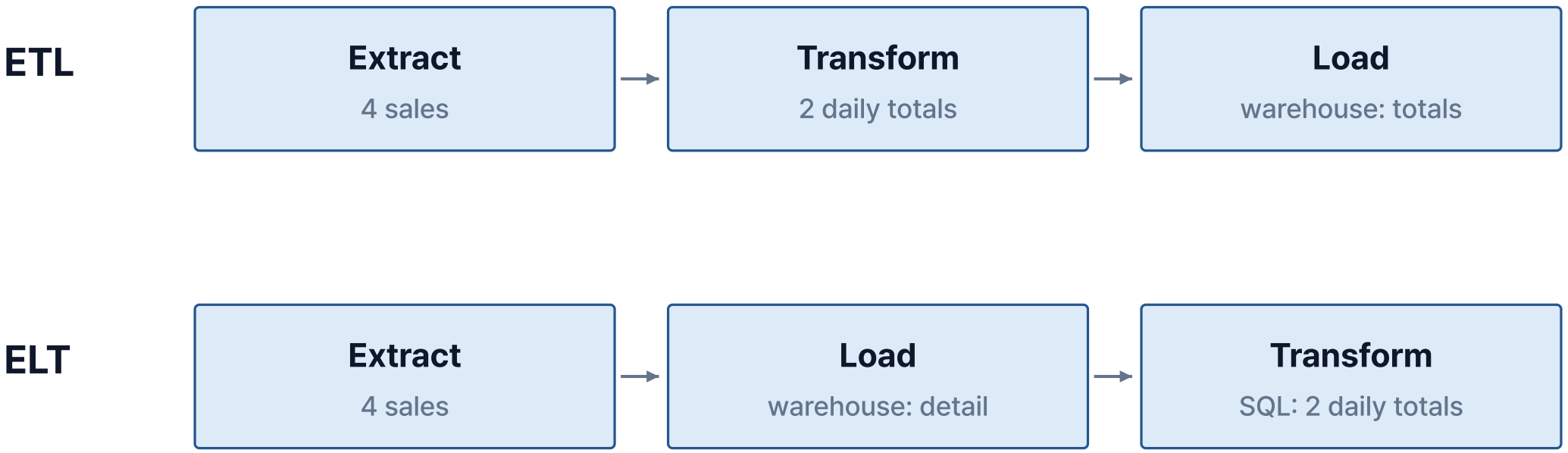
Files can arrive first, but queries still need structure, types and meaning



The string "400" needs an explicit numeric interpretation before summing

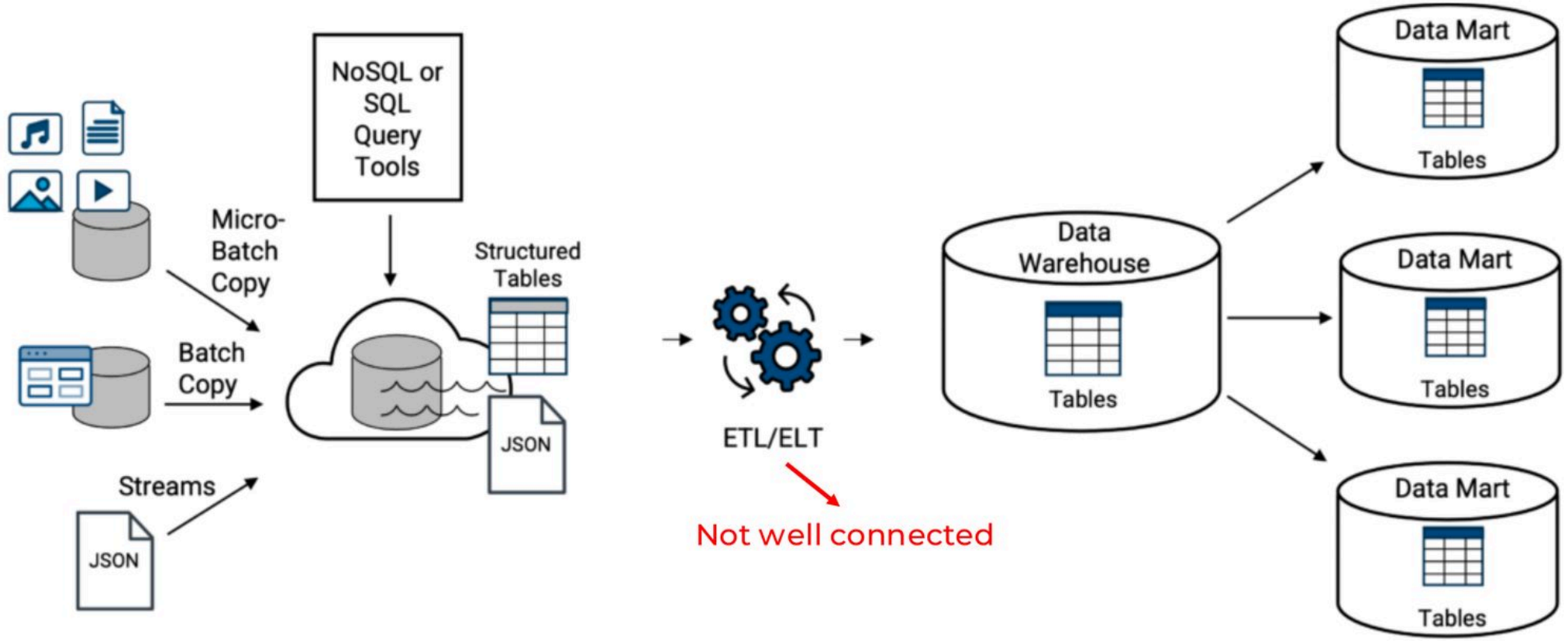
ETL and ELT

Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT)



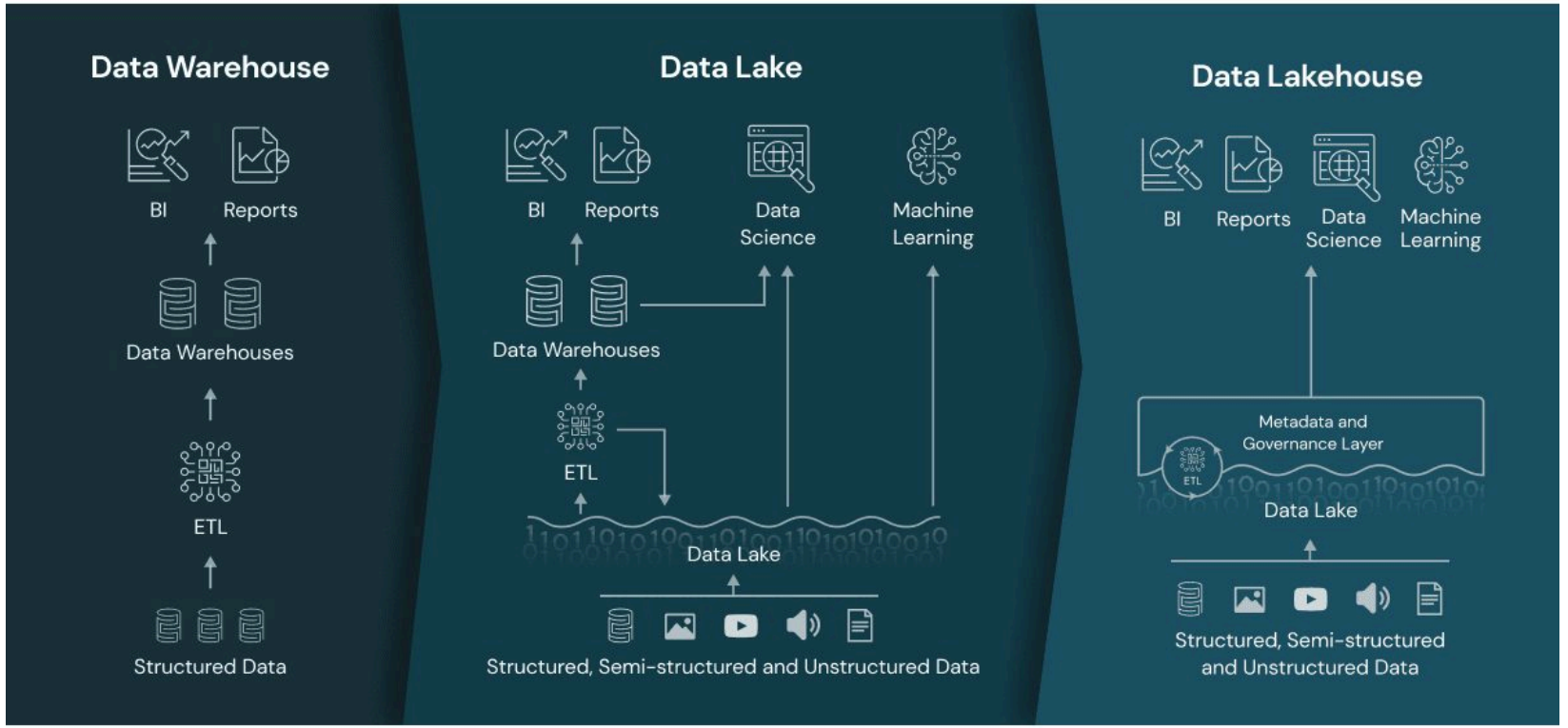
Transform before loading, or load detail first and transform inside the warehouse

From Lake to Warehouse



Separate raw and analytical stores introduce copying, transformations and refresh dependencies

Why a Lakehouse



Lakehouse adds table management over lake files for shared analytical use

Consistent Table Snapshots

Snapshot 1

```
October 2   file_A.parquet
October 3   file_B.parquet
```

`500 + 500 = 1000`

Snapshot 2

```
October 2   file_A2.parquet
October 3   file_B.parquet
```

`550 + 500 = 1050`

Readers see the old or the new complete file list, never a mix

Tools Around a Lakehouse

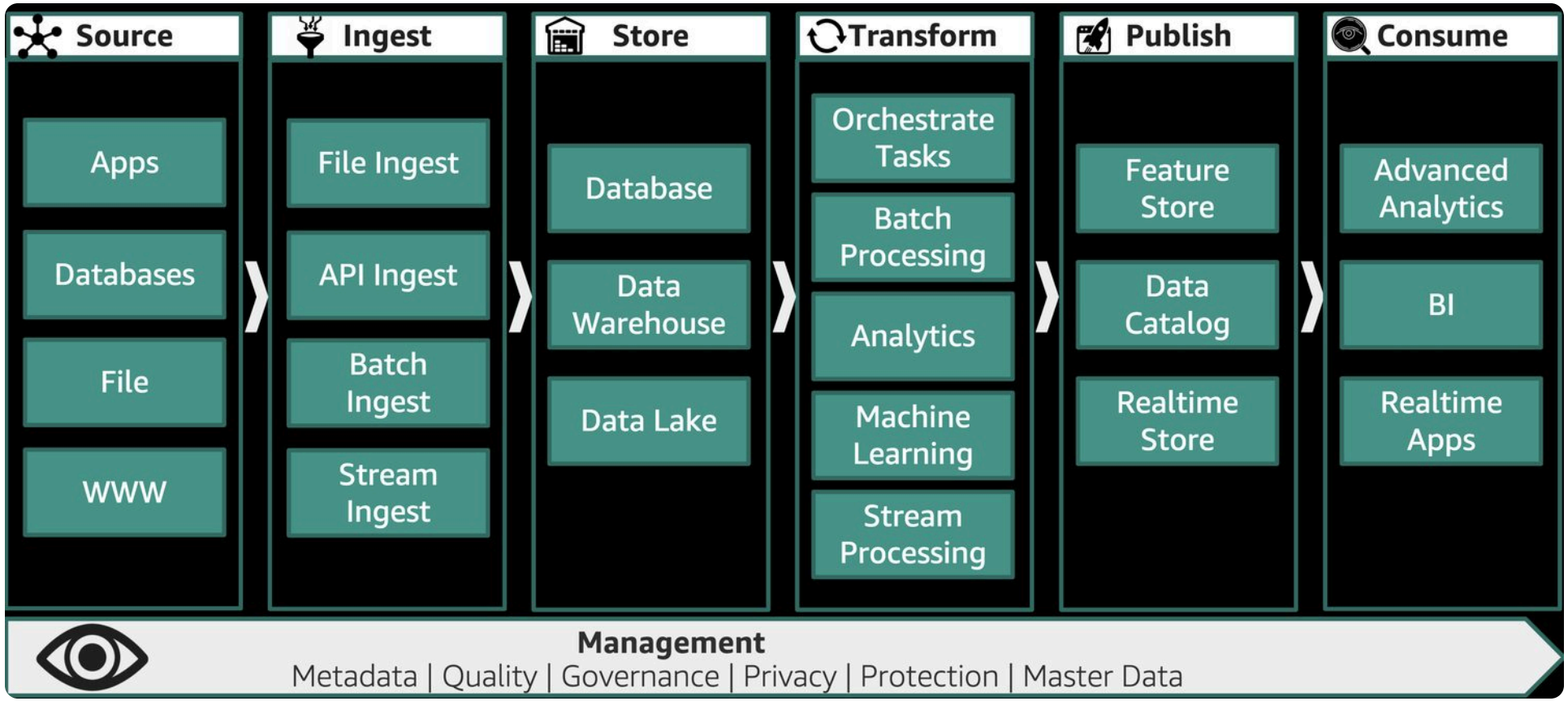
Compute and platform	**Spark, Databricks**	run transformations and queries
Table format	**Iceberg, Delta Lake**	manage table metadata and changes
Columnar file format	**Parquet**	encode values in files
Object storage	**S3, Cloud Storage**	store the data files

Storage, file format, table format and compute belong to different layers

04 · Data Orchestration

01	Cloud and Serverless	Who operates the infrastructure?
02	Analytical Warehouses	How are many rows read and summarized?
03	Lakes and Lakehouses	Where do files become tables?
04	**Data Orchestration**	Which task runs after which, and safely again?

Why Data Orchestration



A successful query can still run before its required data is ready

Airflow at Airbnb

2014
started at Airbnb by M. Beauchemin

2015
announced as open source

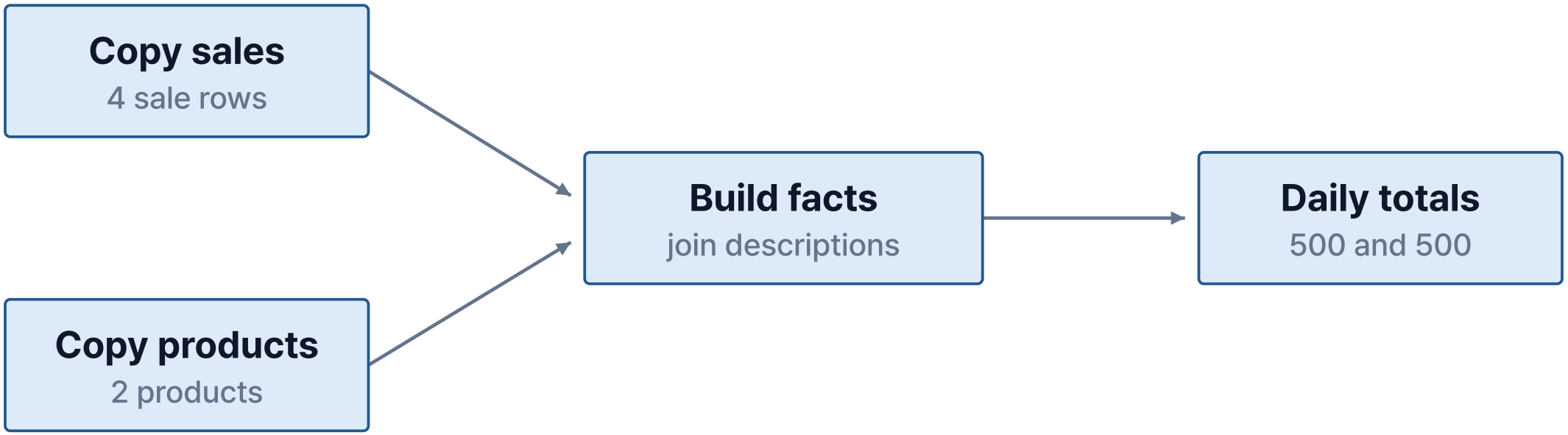
2016
Apache incubation

2019
Apache top-level project

Airflow grew from a company workflow tool into an Apache project

A Sales Workflow DAG

A Directed Acyclic Graph (DAG) defines tasks and their dependencies



Sales and products can load independently; the join waits for both inputs

Dependencies in Python

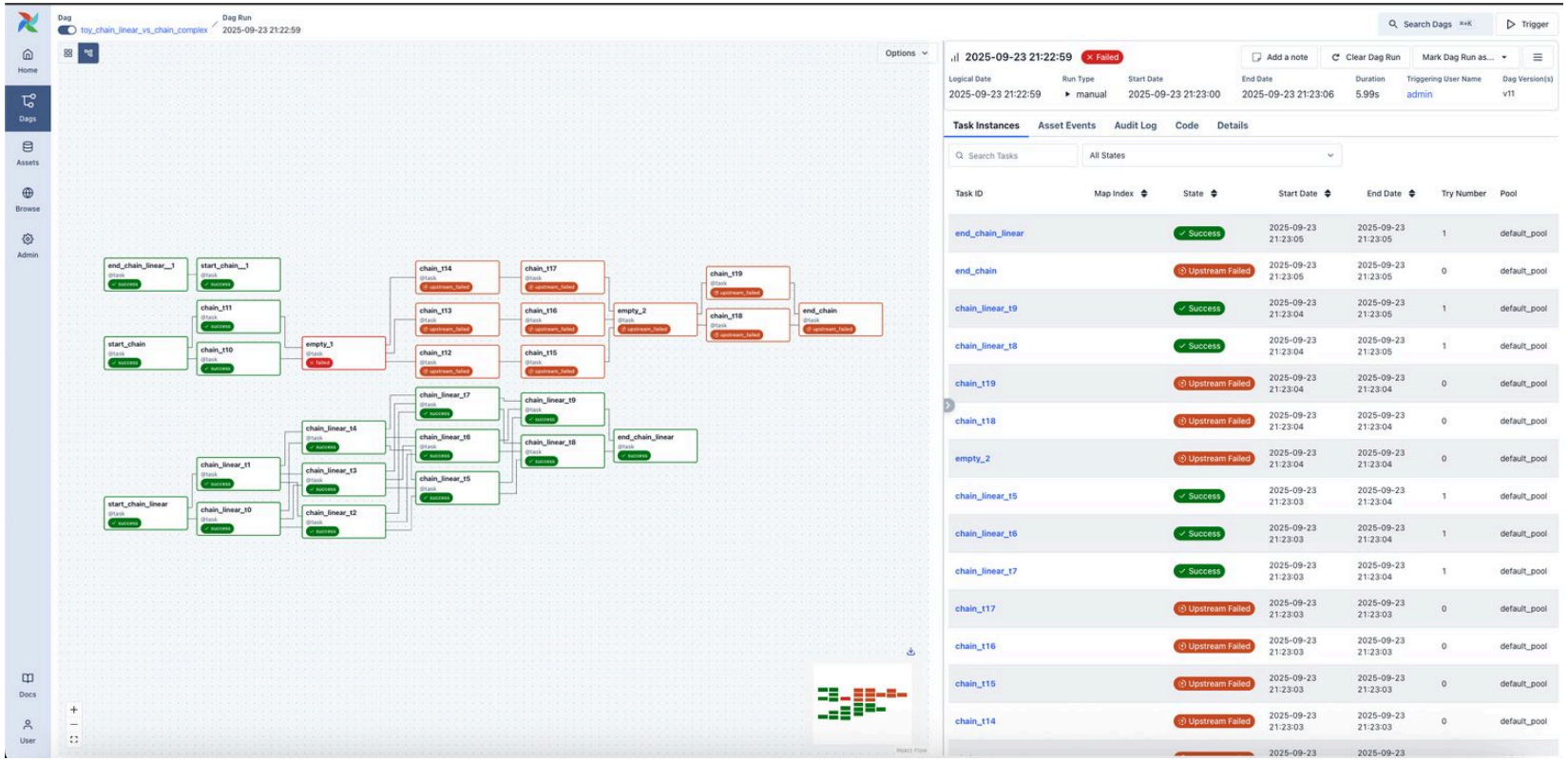
Airflow dependency notation, with task definitions omitted

```
with DAG("daily_sales", schedule="@daily"):
    copy_sales >> build_facts
    copy_products >> build_facts
    build_facts >> daily_totals
```

Both copy tasks are upstream of build_facts

An Airflow Run

Apache Airflow interface



A real run shows succeeded, failed and waiting tasks

Caveat: An Upstream Failure



A failed product load prevents the dependent join from using incomplete inputs

Caveat: Retrying an Append

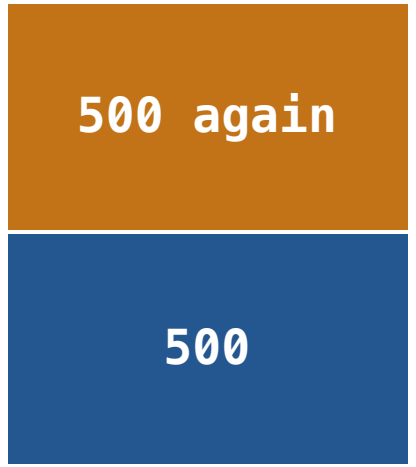
Append once

500



Append twice

1000



Retrying the same append can count the same sales twice

Demo: Repeatable Date Load

Local SQLite exercise: safe task behavior under retries

1. Run the supplied date replacement twice in one script
2. Compare it with appending the same date twice

```
BEGIN;
DELETE FROM daily_sales WHERE
sale_date='2025-10-03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY
sale_date;
COMMIT;
SELECT * FROM daily_sales ORDER BY sale_date;
```

QUERY RESULT

sale_date	total
2025-10-02	500
2025-10-03	500

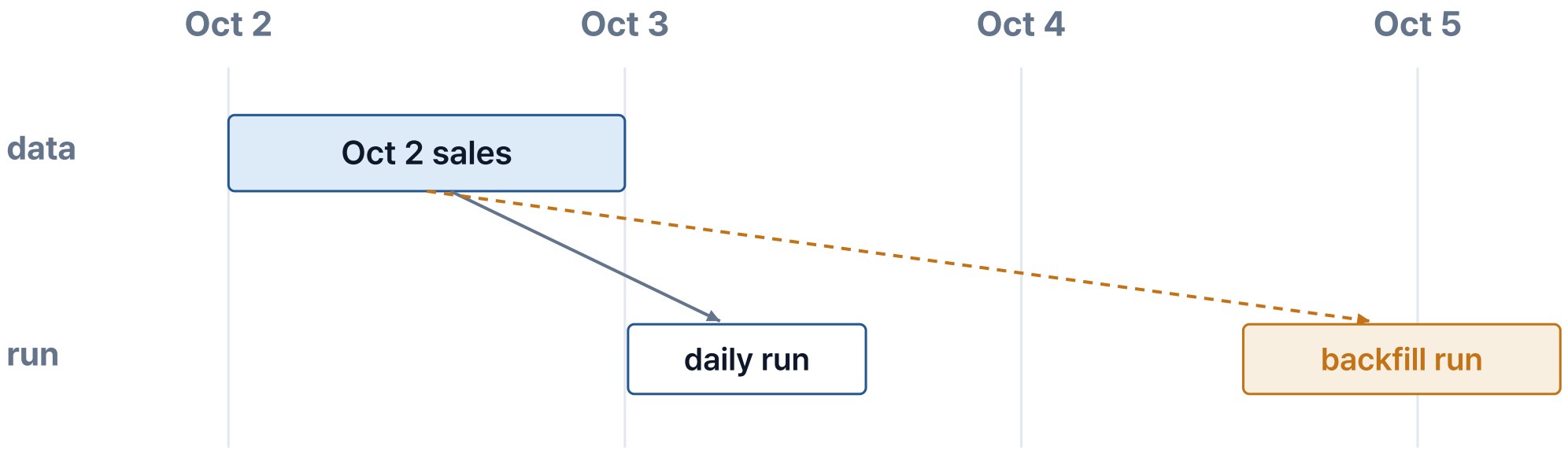
Answer: Repeatable Date Load

```
BEGIN;
DELETE FROM daily_sales WHERE sale_date='2025-10-
03';
INSERT INTO daily_sales
SELECT sale_date,SUM(amount) FROM fact_sales
WHERE sale_date='2025-10-03' GROUP BY sale_date;
COMMIT;
-- run the same block again
SELECT * FROM daily_sales ORDER BY sale_date;
```

sale_date	total
2025-10-02	500
2025-10-03	500

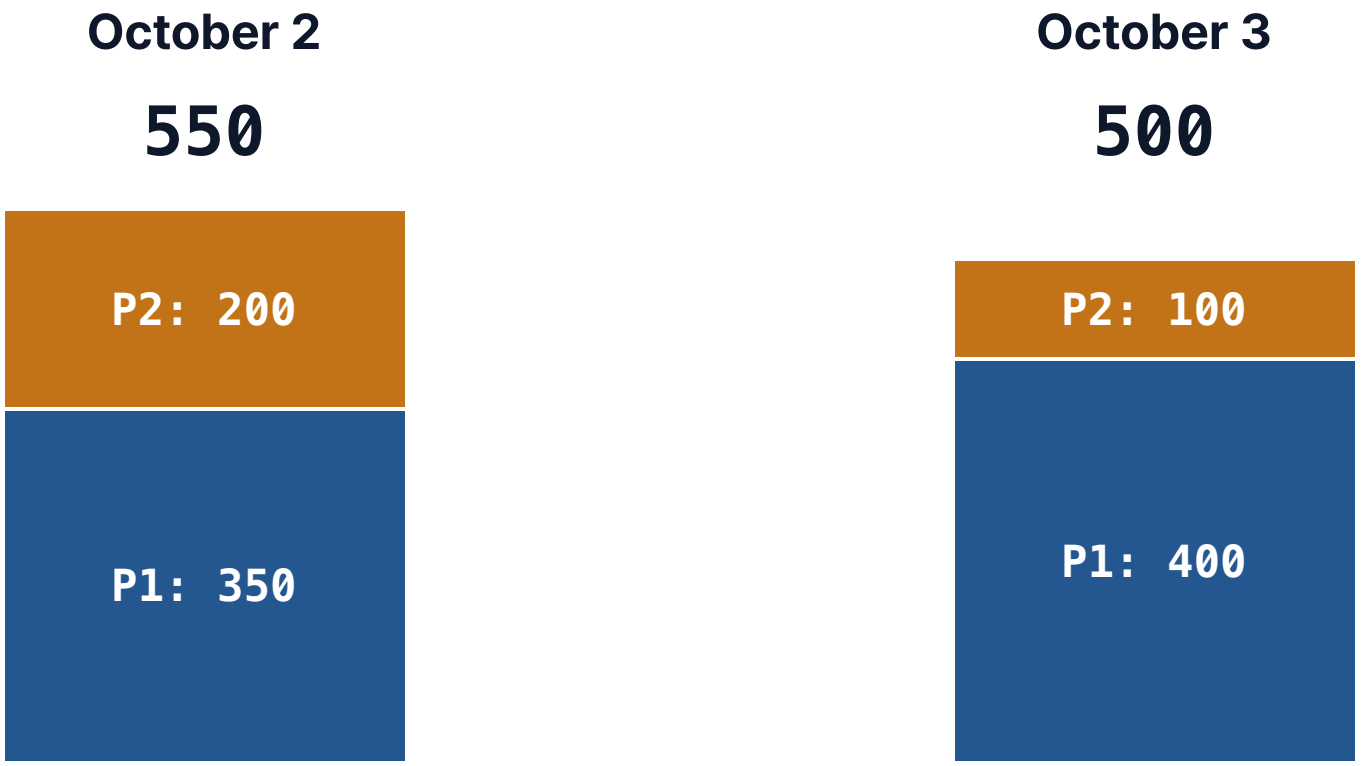
Idempotent load: rerunning the same date range leaves the same result

Data Interval and Run Time



The date being recomputed can differ from the time when the task runs

Backfill After a Correction



Recomputing October 2 changes its total to 550 while October 3 remains 500

Success and Data Checks

Check	Expected for this report	Failure caught
One row per sale_date	2 distinct dates, 2 rows	Repeated date rows
Reconcile to source	500 for each date	Missing or repeated sales
Publish after checks	Both checks succeed	A successful query with wrong data

Execution success and valid analytical data are separate conditions

Takeaway: Tools Have Different Responsibilities

1. **Cloud and serverless** move infrastructure operations behind managed interfaces
2. **Warehouses and lakehouses** organize data for reliable analytical access
3. **Orchestration** connects tasks, run states and safe reprocessing